

Sesión 22052025

Módulo 5

Mayo 2025

1/ Apache Spark

Apache Spark es un motor unificado de cálculo en memoria y un conjunto de bibliotecas para procesamiento paralelo y distribuido de datos en clústeres de ordenadores.

01

Paralelo

Spark distribuir los datos entre varios nodos, en lugar de depender de un solo nodo para procesar los datos.

02

Unificado

El motor es único e independiente de cómo utilicemos Spark:

- Desde la API de DataFrames (para lenguaje R, Python, Java y Scala).
- Desde herramientas externas que lanzan consultas SQL contra Spark etc...

03

En memoria

Todos los cálculos se llevan a cabo en memoria y solo se escriben resultados a disco (parciales o finales) cuando el usuario lo indica explícitamente (operación de guardado) o cuando la operación indicada por el usuario requiere forzosamente realizar movimiento de datos entre nodos (shuffle, el cual se lleva a cabo desde el disco duro del nodo emisor al disco duro del nodo receptor).

04

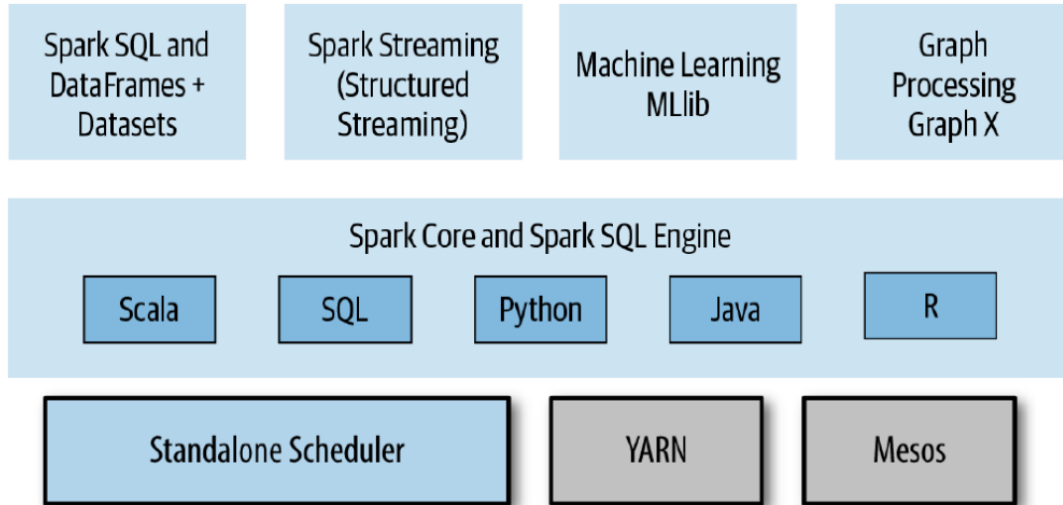
Open Source

Pertenece a la fundación Apache



Apache Spark nació en 2009 como parte de un proyecto de investigación de la Universidad de Berkeley. Un proyecto colaborativo entre profesores, estudiantes e investigadores que se centraba en la creación de aplicaciones para un uso intensivo en el análisis de datos.

Componentes Apache Spark



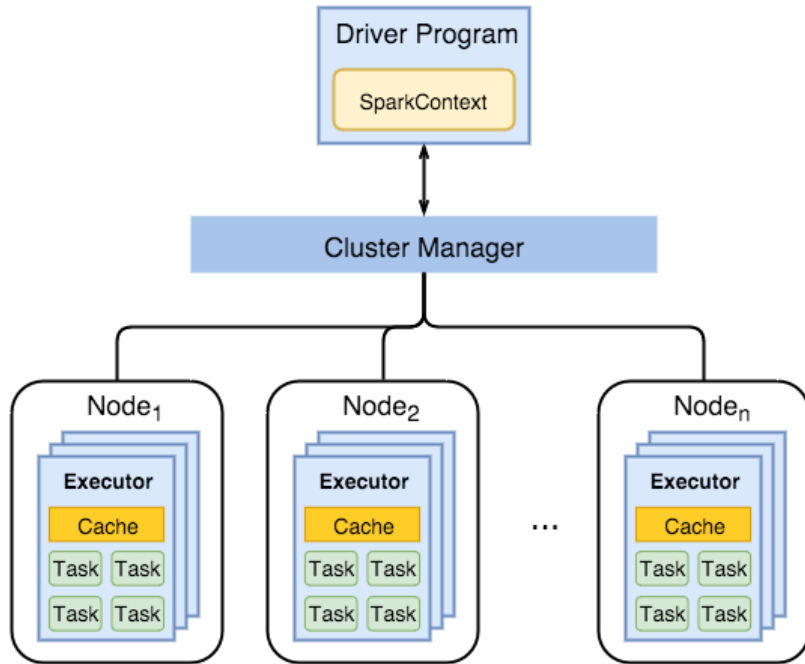
Spark ofrece distintas API para cuatro lenguajes: **Java**, **Scala**, **Python** (paquete de Python **PySpark**, muy usado por científicos de datos, con una API casi idéntica a la de Scala que, en realidad, no es más que una capa adicional que termina llamando al código de Scala) y **R** (paquete de R **SparkR**, con una API muy diferente al resto, más cercana a los comandos típicos de R).

[Tipos de instalación de Apache Spark - Máster en Big Data e Inteligencia Artificial Málaga \(uma.es\)](#)

Spark se basa en un componente principal o core y sobre él existen ciertos componentes que extienden su uso. Los más reseñables serían los siguientes:

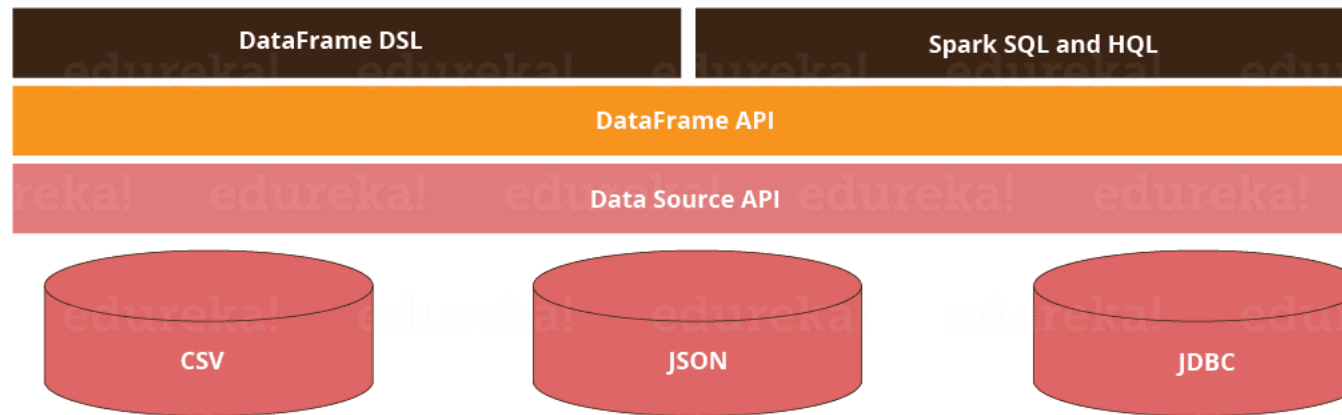
- Spark SQL: Nos permite acceder de forma estructurada a los datos e integrar Spark con Hive, ODBC, JDBC y herramientas de BI.
- Spark Streaming: Ofrece soporte para procesamiento en casi tiempo real a través de un sistema de empaquetamiento de pequeños lotes.
- MLlib: Incluye una biblioteca de algoritmos de machine learning, preparados para ser ejecutados sobre Spark.
- GraphX: Nos ofrece un API para computación paralela de grafos.

Componentes Apache Spark



El componente principal es el módulo Spark Core. Es la base o conjunto de librerías donde se apoya el resto de módulos. Es el núcleo del framework. En él se encuentran las estructuras de datos fundamentales de Spark, tales como los RDD (Resilient Distributed Dataset), y las operaciones que se puede llevar a cabo con él. Spark Core es el corazón de Apache Spark y es responsable de proporcionar transmisión de tareas distribuidas, programación y funcionalidad de E/S. El motor Spark Core utiliza el concepto de conjunto de datos distribuido resistente (RDD) como tipo de datos básico. En última instancia, todas las operaciones que se pueden hacer con Spark, independientemente del módulo utilizado o la API donde se encuentren, se traducen automáticamente por parte de Spark a operaciones sobre RDD, que son las únicas que sabe ejecutar el motor.

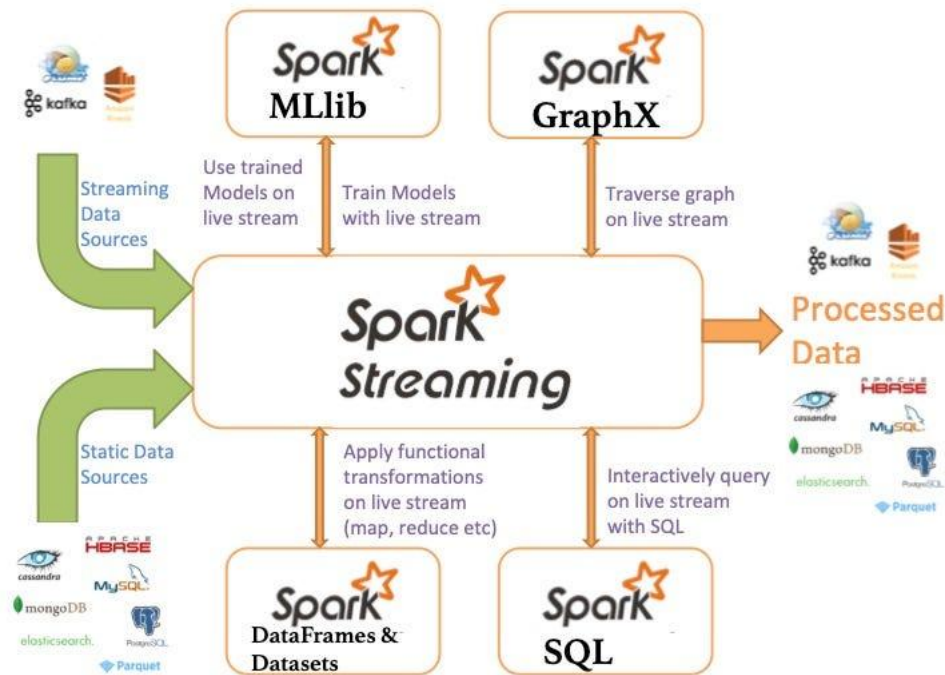
Architecture Of Spark SQL



Spark SQL es el módulo de Spark que permite utilizar datos estructurados. Ofrece un método común para acceder a fuentes de datos diversas. Gracias a este módulo, puedes consultar datos estructurados de programas de Spark con SQL o con la API de DataFrame que te resulte más cómoda. Spark SQL admite la sintaxis de HiveQL y franquea el acceso a almacenes de Apache Hive. El modo de servidor proporciona conectividad estándar mediante JDBC u ODBC.

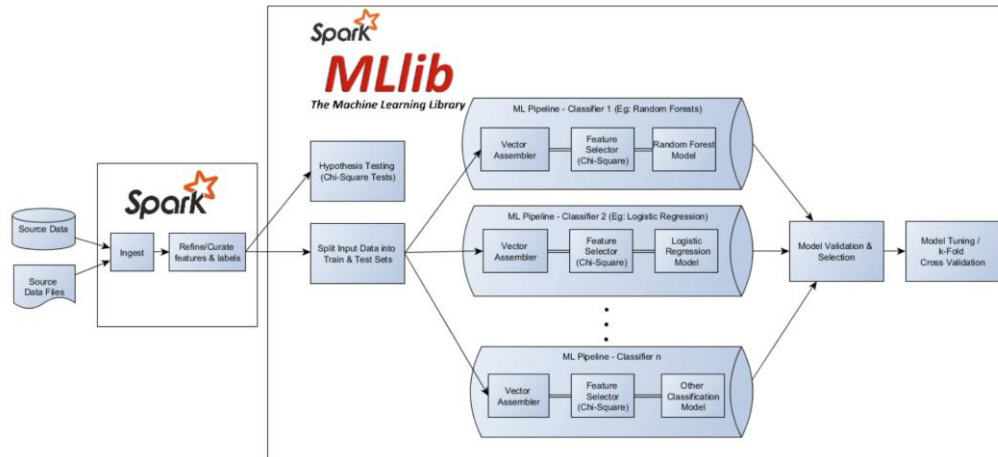
[Spark SQL & DataFrames | Apache Spark](#)

Componentes Apache Spark



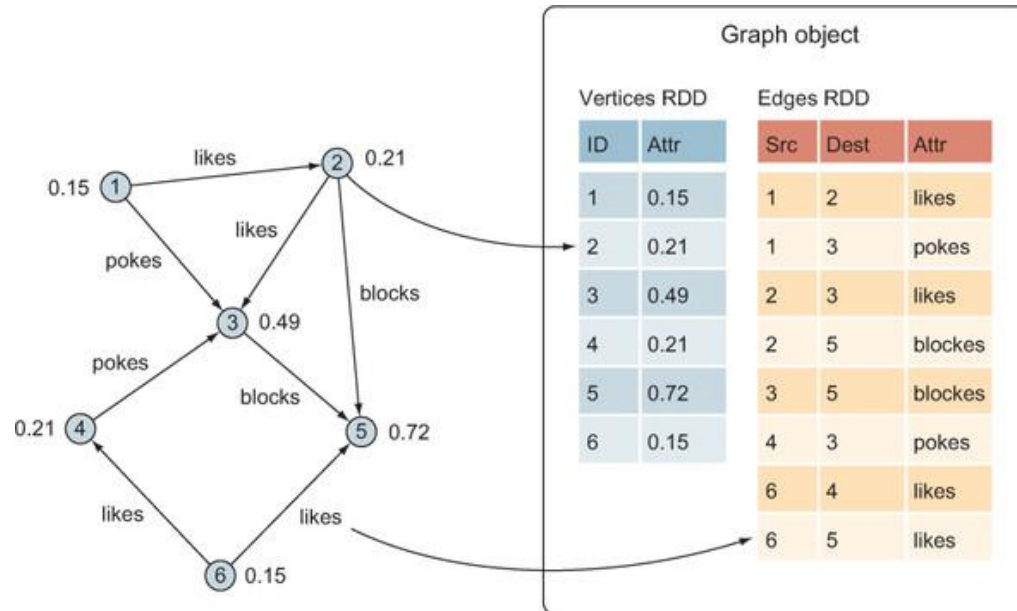
Spark Streaming facilita la creación de soluciones de streaming escalables y tolerantes a fallos. Como incorpora la API con integración de lenguajes de Spark al procesamiento de streaming, puedes escribir tareas de streaming igual que lo haces con las tareas por lotes. Spark Streaming no solo admite Java, Scala y Python, sino que incluye semántica de una sola vez y con reconocimiento del estado que está lista para utilizarse.

[Spark Streaming - Spark 3.5.1 Documentation \(apache.org\)](https://spark.apache.org/docs/3.5.1/streaming.html)



MLlib es una biblioteca básica de Spark que proporciona numerosas utilidades para las tareas de ML como por ejemplo:

- Clasificación
- Regresión
- Clusterización
- Modelado
- Descomposición en valores singulares (SVD) y Análisis de los componentes principales (PCA)
- Comprobación de hipótesis y cálculo de estadísticas de ejemplo



Spark GraphX en Apache Spark proporciona el procesamiento de **grafos distribuidos** dentro de la arquitectura de este sistema de computación. Para ello, debes saber que **un grafo se compone de *vertex* y *edge***.

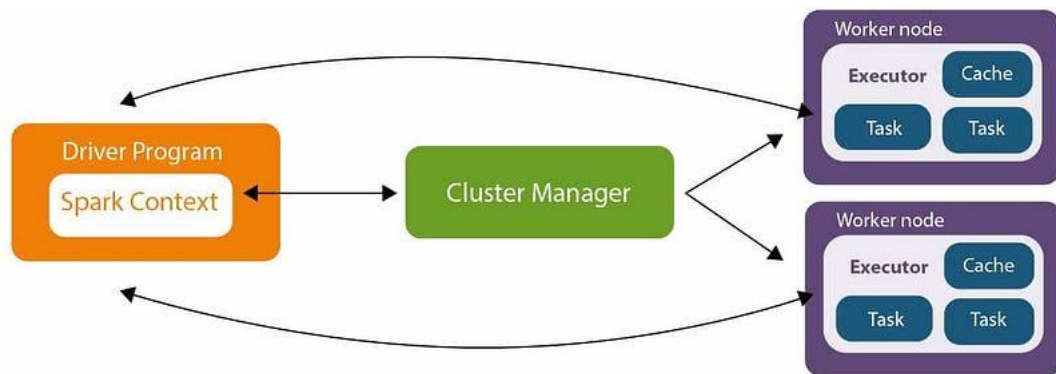
En efecto, este es uno de los fundamentos de *Spark Stack*, ya que **te permite trabajar con estas herramientas para la creación de grafos distribuidos**:

- **Vértices:** estos son, literalmente, los vértices existentes, es decir, **las propiedades que se establecen como un punto de inflexión en los grafos**.
- **Edges:** hacen referencia a las relaciones existentes. Se muestran como **las relaciones entre un vértice y otro** según los datos procesados.
- **Triplets:** se conoce así a la **relación entre dos vértices y su *edge***.

[GraphX | Apache Spark](#)



Arquitectura Apache Spark



No debemos olvidar que, al escribir una aplicación en Spark, en realidad, estamos escribiendo una aplicación secuencial (no paralela) utilizando la biblioteca de Spark para el lenguaje en el que estemos programando (Java, Scala, Python o R). Cuando ejecutamos el código de este programa (proceso que se denomina driver), lo hacemos sobre una máquina concreta, que puede ser interna o externa al clúster. Por ejemplo, podríamos escribir un programa en Spark y ejecutarlo en nuestro ordenador portátil, y, desde aquí, el programa driver podría conectarse a un clúster de Spark remoto.

Todos los componentes de Spark, incluidos los procesos Controlador, Maestro y Ejecutor, se ejecutan en máquinas virtuales Java (JVM).

Una JVM es un motor de ejecución multiplataforma que puede ejecutar instrucciones compiladas en código de bytes de Java.

Scala, en el que está escrito Spark, se compila en código de bytes y se ejecuta en JVM.

En el transcurso de la ejecución, normalmente, necesitaremos crear un objeto denominado `sparkSession`, de la clase `SparkSession` de Spark. En el momento de crear este objeto, hay que indicar dónde (en qué dirección IP y en qué puerto) existe un clúster de Spark configurado. Ciertas aplicaciones, como, por ejemplo, Jupyter Notebook (cuando está configurado para Spark) o la línea de comandos (intérprete) de Spark, tanto en Scala (`spark-shell`) como en Python (`pyspark-shell`), ya nos dan un objeto `sparkSession` creado al arrancar la aplicación. No obstante, al iniciar el shell, debemos pasarle la configuración relativa al clúster para que la `sparkSession` se cree correctamente.

De lo contrario, se estará ejecutando en local, sin conexión con un clúster. De esta manera, la `sparkSession` establece comunicación con el gestor de clúster para poder enviar tareas a los workers (nodos del clúster disponibles para Spark). A partir de este momento, podemos utilizar dichos recursos para las sentencias que requieran ejecución distribuida.

Es importante recalcar esto último: la ejecución de un programa en Spark es secuencial, en una sola máquina, tal como sería cualquier otro programa en el lenguaje elegido, excepto cuando el flujo de programa llega a ciertas funciones específicas de Spark que desencadenan ejecución distribuida, que son la mayoría (pero no todas) de las que forman parte de la API de Spark. Muchas se aplican sobre el objeto `sparkSession` y otras sobre estructuras de datos distribuidas que hayamos ido creando en el programa, como, por ejemplo, RDD o dataframes de Spark.

¿Como se ejecuta un job?

¿Qué es un Job en Spark? Un job en Spark es la unidad de trabajo más grande. Es una secuencia de transformaciones (map, filter, etc.) que se aplican a un RDD (Resilient Distributed Dataset) y que culminan en una acción (count, collect, etc.).

Componentes Clave y su Rol:

•**Driver:**

- Es el programa principal que envía el job al cluster.
- Crea el SparkContext, que es el punto de entrada a la programación de Spark.
- Coordina la ejecución del job y recibe los resultados.

•**SparkContext:**

- Representa una conexión con un cluster Spark.
- Crea RDDs y distribuye los datos a través del cluster.
- Es responsable de la creación del DAG (Directed Acyclic Graph) de ejecución.

•**SparkSession:**

- Es la entrada unificada para Spark SQL, DataFrames y Datasets en Spark 2.0 y versiones posteriores.
- Simplifica la creación de SparkContext y proporciona funcionalidades adicionales.

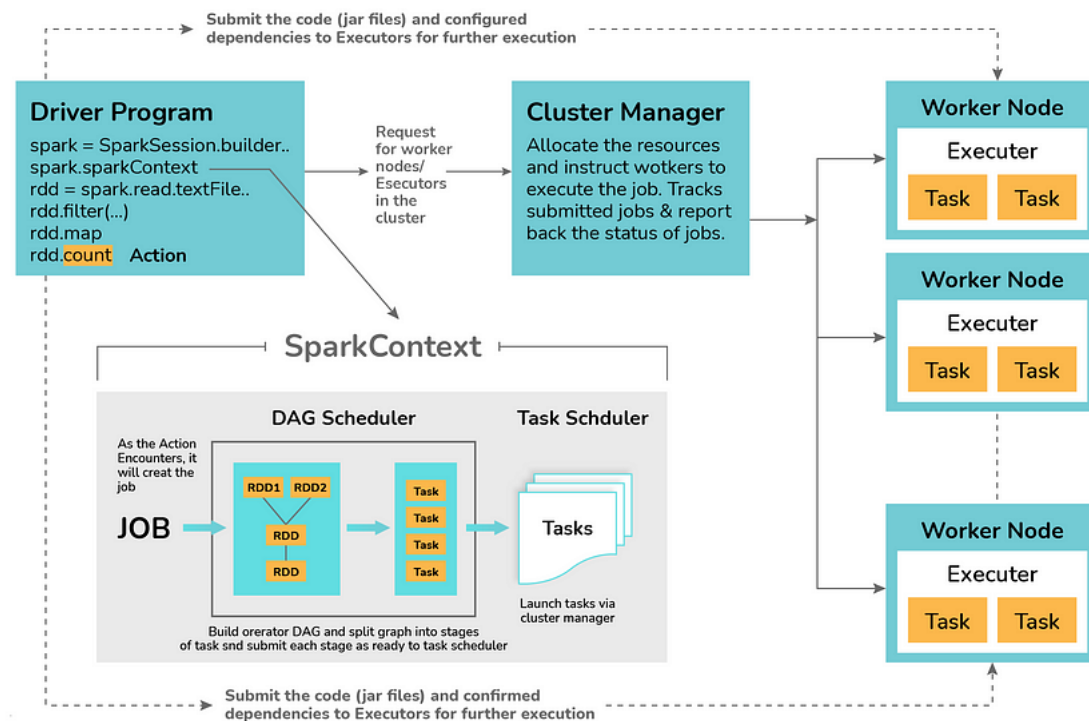
•**Executors:**

- Son procesos que se ejecutan en los nodos del cluster.
- Ejecutan las tareas asignadas por el driver.
- Tienen acceso a los datos locales y realizan los cálculos.

•**DAG (Directed Acyclic Graph):**

- Es una representación gráfica de las transformaciones que se aplican a un RDD.
- El DAG es optimizado antes de su ejecución.

Internals of Job Execution In Spark



¿Comos se ejecuta un job?

Pasos en la Ejecución de un Job:

1.Creación del SparkContext o SparkSession:

El driver crea un SparkContext o SparkSession para conectarse al cluster.

2.Creación de RDDs:

Se crean RDDs a partir de fuentes de datos (HDFS, archivos locales, bases de datos, etc.).

3.Aplicación de Transformaciones:

Se aplican transformaciones a los RDDs para crear nuevos RDDs (map, filter, join, etc.).

Cada transformación genera una nueva etapa en el DAG.

4.Creación del DAG:

Spark construye el DAG basado en las transformaciones aplicadas.

5.Optimización del DAG:

El DAG es optimizado para reducir la cantidad de datos que se mueven entre los nodos del cluster.

6.Envío del DAG al Cluster:

El driver envía el DAG a los executors.

7.Ejecución de las Tareas:

Los executors dividen el trabajo en tareas y las ejecutan.

Las tareas se ejecutan en paralelo en los diferentes nodos del cluster.

8.Agrupación de los Resultados:

Los resultados de las tareas son agrupados y enviados al driver.

9.Retorno de los Resultados:

El driver devuelve los resultados al programa principal.

```
# Crear un SparkSession
```

```
spark =
```

```
SparkSession.builder.appName("MiPrimerJob").getOrCreate()
```

```
# Crear un RDD a partir de una lista
```

```
rdd = spark.sparkContext.parallelize([1, 2, 3, 4, 5])
```

```
# Aplicar transformaciones
```

```
rdd_multiplicado = rdd.map(lambda x: x * 2)
```

```
rdd_filtrado = rdd_multiplicado.filter(lambda x: x > 4)
```

```
# Acceder a los resultados (acción)
```

```
resultado = rdd_filtrado.collect()
```

```
print(resultado)
```

- Un job en Spark es una secuencia de operaciones sobre RDDs.
- El driver coordina la ejecución del job y los executors realizan el trabajo.
- El DAG representa el plan de ejecución del job.
- La evaluación perezosa permite optimizar el job antes de ejecutarlo

Arquitectura Apache Spark

SparkContext

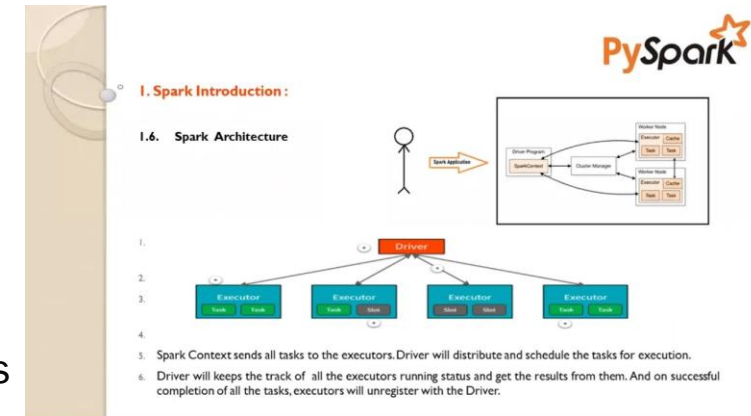
Imagina que quieres empezar a trabajar con Spark. El primer paso es crear un **SparkContext**. Piensa en él como la puerta de entrada a tu clúster de Spark. Es el objeto que te permite:

- **Conectarte a un clúster:** Ya sea local en tu computadora o en un clúster más grande.
- **Crear RDDs:** Los RDDs son la unidad básica de datos en Spark. Son como colecciones de datos distribuidas a través de los nodos del clúster.
- **Ejecutar operaciones:** Una vez que tienes tus RDDs, puedes realizar todo tipo de operaciones sobre ellos, como filtrar, transformar, agrupar, etc.

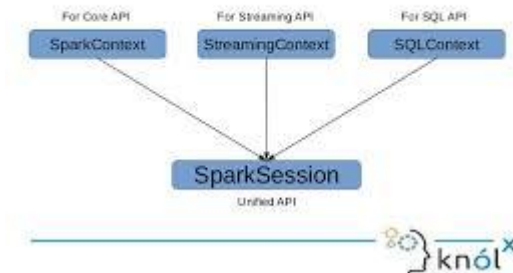
SparkSession

La **SparkSession** es una evolución del SparkContext. Es una forma más moderna y unificada de interactuar con Spark. Combina las funcionalidades del SparkContext con otras capacidades, como:

- **DataFrame API:** Una API más amigable para trabajar con datos estructurados, similar a las tablas de una base de datos.
- **Dataset API:** Una API que combina los beneficios de los RDDs y los DataFrames.
- **SQL:** Puedes ejecutar consultas SQL directamente sobre tus datos.
- **Machine Learning:** SparkSession proporciona herramientas para realizar tareas de machine learning.



What is SparkSession ?





2/ Apache Spark DataFrame

¿Qué son los Spark DataFrames?

- **Analogía con tablas:** Compara un DataFrame con una tabla de una base de datos relacional o una hoja de cálculo. Cada fila representa un registro y cada columna un atributo.
- **Ventajas sobre RDDs:**
 - **Esquema:** Los DataFrames tienen un esquema definido, lo que facilita la manipulación y la optimización de las consultas.
 - **Tipo de datos:** Soporta una amplia variedad de tipos de datos, incluyendo estructuras complejas como arrays y mapas.
 - **API más amigable:** La API de DataFrames es más intuitiva y similar a las herramientas de análisis de datos tradicionales.

```
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder.appName("DataFrameExample").getOrCreate()
```

```
# Crear un DataFrame a partir de una lista
```

```
data = [("Alice", 30), ("Bob", 25), ("Charlie", 35)]
```

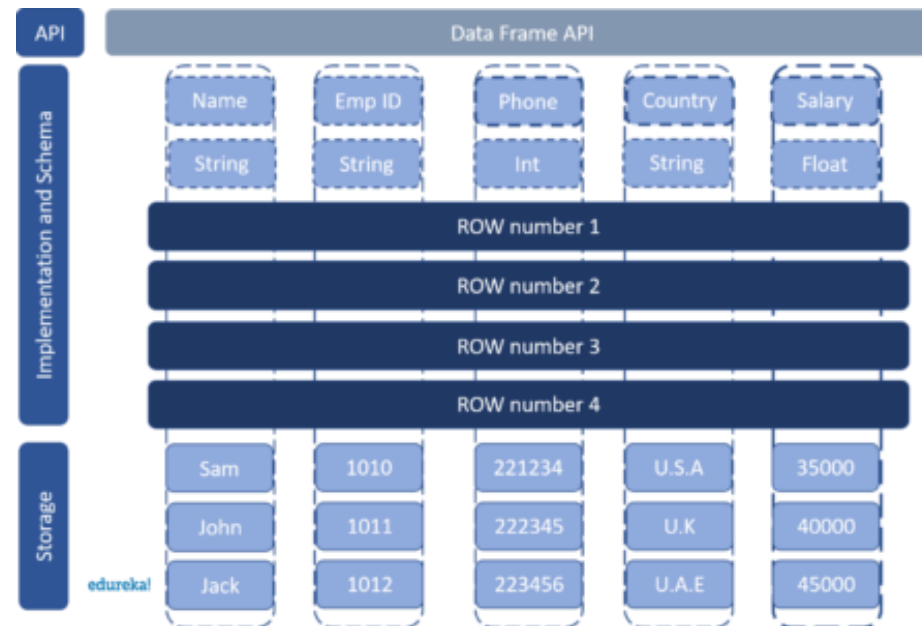
```
df = spark.createDataFrame(data, ["name", "age"])
```


Apache Spark DataFrames

Un dataframe de Spark es una tabla de datos distribuida en la RAM de los nodos, formada por filas y columnas con nombre y tipo (incluidos tipos complejos), similar a una tabla en una base de datos relacional.

Internamente, un dataframe no es más que un RDD de objetos de tipo Row de Spark, cada uno de los cuales representa una fila de una tabla como un vector cuyos componentes (lo que serían las columnas de una tabla) tienen nombre y tipo predefinido. El esquema (schema) de un dataframe define el nombre y el tipo de dato de cada una de estas columnas. Cada dataframe envuelve (tiene dentro) un RDD, al que se puede acceder como el atributo 'rdd'. Ejemplo: `variableDF.rdd`. Por eso, los conceptos de transformación y acción se aplican también a dataframes.

Los dataframes de Spark son un tipo de dato definido por Spark, están distribuidos físicamente y se manejan mediante la API de Spark.



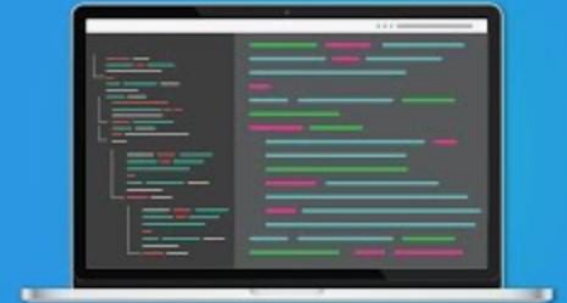
APACHE SPARK DATAFRAME

Part 1

Learn Apache Spark

simplilearn

PYSPARK DATAFRAME TUTORIAL



Apache Spark DataFrame

¿Cómo se crea un DataFrame?

- A partir de RDDs:
 - Convertir un RDD en un DataFrame especificando el esquema.
`rdd = sc.parallelize([("Alice", 30), ("Bob", 25)])`
`df = spark.createDataFrame(rdd, ["name", "age"])`
- A partir de archivos: Leer archivos CSV, JSON, Parquet, etc. utilizando las funciones `read.csv`, `read.json`, `read.parquet`
`df = spark.read.csv("data.csv", header=True, inferSchema=True)`
- A partir de bases de datos: Conectar a bases de datos relacionales (MySQL, PostgreSQL) o almacenes de datos (Hive) utilizando los conectores correspondientes.

```
df = spark.read.jdbc("jdbc:postgresql://localhost:5432/mydatabase", "mytable")
```

Last run took 2 mins 51.52 secs on May 13 2020 12:28:30 PM by [redacted] on spark cluster

```
[1]: sdf = spark.sql("select * from default_qubole_airline_origin_destination limit 10")
display(sdf)
```

Kernel Id: 89723802-f6af-4dfa-8863-c815790058fc
Starting Spark application

ID	Name	YARN Application ID	Kind	State	Spark UI	Driver log	Current session?
0	scoped_s_...qubole	application_1589454926444_0001	pyspark	idle	Link	Link	✓

SparkSession available as 'spark'.

► Spark Jobs (1)

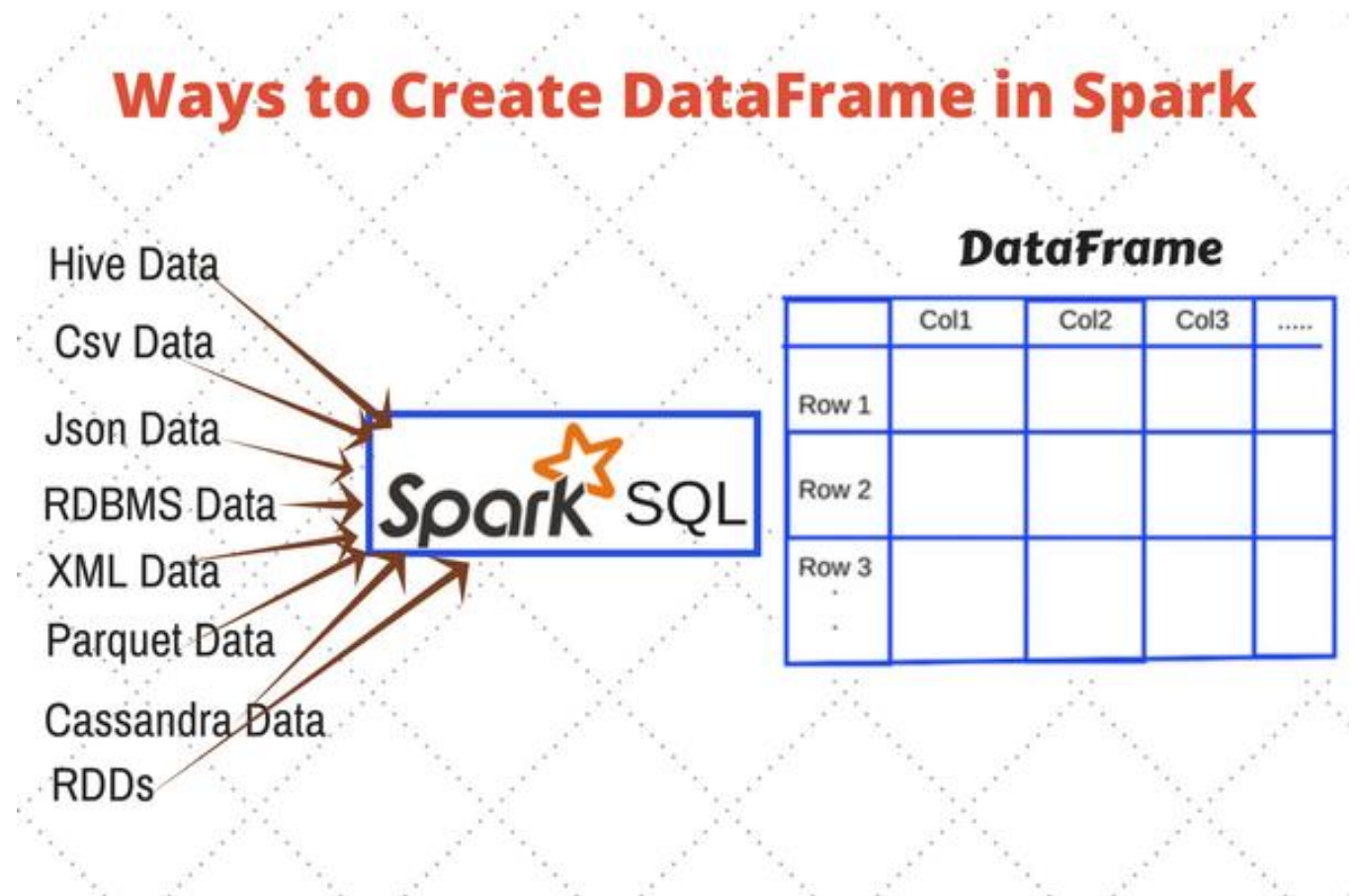
Search...

itinid	mktid	seqnum	coupons	year
"itinID"	"MktID"	"SeqNum"	"Coupons"	"Year"
"200713540007"	"200716345063"	5	6	2007
"200713540007"	"200716345063"	6	6	2007
"200713540008"	"200716345064"	1	5	2007
"200713540008"	"200716345064"	2	5	2007
"200713540008"	"200716345064"	3	5	2007
"200713540008"	"200716345065"	4	5	2007
"200713540008"	"200716345065"	5	5	2007
"200713540009"	"200716345066"	1	3	2007
"200713540009"	"200716345066"	2	3	2007



Apache Spark DataFrames : Fuentes de origen

- **HDFS.** Spark puede leer de HDFS diversos formatos de archivo: CSV, JSON, Parquet, ORC y texto plano. No obstante, la comunidad de desarrolladores ha proporcionado mecanismos para leer otros tipos de ficheros, como los XML, entre otros. La terminación indicada en el nombre de archivo no informa a Spark de nada, solo puede servir como pista al usuario que vaya a leer el fichero.
- **Amazon S3.** Almacén de objetos distribuido creado por Amazon, de donde Spark también puede leer cualquiera de los formatos de fichero nombrados anteriormente.
- **Bases de datos relacionales,** mediante conexiones JDBC u ODBC. Spark es capaz de leer en paralelo a través de varios *workers* conectados simultáneamente a una base de datos relacional, cada uno de los cuales lee porciones diferentes de una misma tabla. Cada porción va a una partición del resultado. Spark puede enviar una consulta a la base de datos y leer el resultado como DataFrame.
- Conectores para **bases de datos no relacionales.** Los conectores son específicos, desarrollados por la comunidad o por el fabricante (por ejemplo: Cassandra, MongoDB, HBase ElasticSearch...).
- Cola distribuida **Kafka** (que veremos en un tema posterior). Para datos que se van leyendo de un *buffer*.
- También datos que llegan en **streaming a HDFS** (ficheros que se van creando nuevos en tiempo real).



En general, para leer datos desde Spark, se usa un atributo de la SparkSession, spark.read, y se especifican diferentes opciones, dependiendo del tipo de fichero por leer:

- **Formato.** Tal y como se mencionó anteriormente, el formato de fichero puede ser CSV, JSON, Parquet, Avro, ORC, JDBC/ODBC o texto plano, entre otros muchos.

- **Esquema.** Hay cuatro opciones referentes al esquema:

1. Algunos tipos de fichero almacenan el esquema junto a los datos (por ejemplo, Parquet, ORC o Avro) y, por tanto, no es necesario indicar ningún esquema adicional.
2. Para los tipos de fichero que no almacenan el esquema, es posible solicitar a Spark que trate de inferirlo con la opción inferSchema activada (true). Hay que tener en cuenta que esta opción conllevará más tiempo de lectura, dado que Spark necesita leer una serie de líneas del fichero y analizarlas para tratar de adivinar el esquema.
3. Podemos pedir a Spark que no trate de inferir el esquema. En este caso, todos los datos se leerán como si fueran texto (*string*).

Leer CSV

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Create DataFrame").getOrCreate()

df = spark.read.format("csv").options(header='true', inferSchema='true', delimiter=',')
df.show()
```

Leer JSON

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("Create DataFrame").getOrCreate()

df = spark.read.json("path/to/file.json")

df.show()
```

Apache Spark DataFrames : Inferir esquema

Cabe recordar una vez más que la opción `inferSchema` indica a Spark que trate de adivinar el tipo de cada columna, en cuyo caso Spark tratará de inferir lo mejor posible esta información. No obstante, puede darse el caso de que, en una columna que debería ser de tipo entero, falte alguna información y de que Spark, ante la duda, infiera que dicha columna es de tipo texto (*string*). Para evitar este tipo de incorrecciones, es mejor especificar explícitamente el esquema si se conoce de antemano. Un esquema en Spark no es más que un objeto `StructType`, compuesto por un conjunto de `StructFields`. Cada `StructField` representa un registro (columna) de una fila; por tanto, se compone de un nombre (`name`), un tipo (`type`), un booleano que indica si la columna puede contener datos null (es decir, datos inexistentes), así como otra información opcional. Por ejemplo, podemos definir en `pyspark` el esquema de un fichero JSON donde cada fila contiene tres campos (columnas) de la siguiente forma:

```
from pyspark.sql.types import StructField, StructType, StringType, LongType
fileSchema = StructType([
    StructField("dest_country_name", StringType(), True),
    StructField("origin_country_name", StringType(), True),
    StructField("count", LongType(), False)
])
```

Una vez que se tiene el esquema definido, solo resta leer los datos utilizando dicho esquema:

```
myDF = spark.read.format("json")
    .schema(fileSchema)
    .load("/path/to/file.json") # spark es el objeto SparkSession
```


Tipos de Datos en Spark DataFrames

Los DataFrames en Spark ofrecen una amplia variedad de tipos de datos para representar diferentes tipos de información.

Estos tipos de datos se pueden clasificar en:

Tipos de Datos Básicos

- **ByteType:** Enteros de 8 bits (Byte).
- **ShortType:** Enteros de 16 bits (Short).
- **IntegerType:** Enteros de 32 bits (Integer).
- **LongType:** Enteros de 64 bits (Long).
- **FloatType:** Números de punto flotante de precisión simple.
- **DoubleType:** Números de punto flotante de doble precisión.
- **BooleanType:** Valores booleanos (True o False).
- **StringType:** Cadenas de caracteres.
- **TimestampType:** Marcas de tiempo.
- **DateType:** Fechas.
- **BinaryType:** Datos binarios.

Tipos de Datos Complejos

- **ArrayType:** Arreglos de elementos de un tipo de dato específico.
- **MapType:** Mapas (diccionarios) con claves de un tipo y valores de otro tipo.
- **StructType:** Estructuras que contienen campos con diferentes tipos de datos. Esto permite representar datos jerárquicos.

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType
```

```
schema = StructType([  
    StructField("name", StringType(), True),  
    StructField("age", IntegerType(), True),  
    StructField("address", StructType([  
        StructField("street", StringType(), True),  
        StructField("city", StringType(), True)  
    ])  
])
```

Apache Spark DataFrames : Structure Type/Field

Structure type: PySpark proporciona la clase StructType de pyspark.sql.types para definir la estructura del DataFrame.

StructType representa un esquema, que es una colección de objetos StructField. Un StructType es esencialmente una lista de campos, cada uno con un nombre y tipo de datos, que define la estructura del DataFrame. Permite la creación de estructuras anidadas y tipos de datos complejos.

StructField representa un campo en el esquema que contiene metadatos como el nombre, el tipo de datos y el estado anulable del campo. Cada objeto StructField define una sola columna en el DataFrame, especificando su nombre y el tipo de datos que contiene.

```
L:import pyspark
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

data = [("James", "", "Smith", "36636", "M", 3000),
        ("Michael", "Rose", "", "40288", "M", 4000),
        ("Robert", "", "Williams", "42114", "M", 4000),
        ("Maria", "Anne", "Jones", "39192", "F", 4000),
        ("Jen", "Mary", "Brown", "", "F", -1)
]

schema = StructType([ \
    StructField("firstname", StringType(), True), \
    StructField("middlename", StringType(), True), \
    StructField("lastname", StringType(), True), \
    StructField("id", StringType(), True), \
    StructField("gender", StringType(), True), \
    StructField("salary", IntegerType(), True) \
])

df = spark.createDataFrame(data=data, schema=schema)
df.printSchema()
df.show(truncate=False)
```


Operaciones Básicas

- Selección de columnas:

df.select("name", "age")

- Filtrado de filas:

df.filter(df.age > 30)

- Agrupación y agregación

df.groupBy("name").count()

- Uniones:

df1.join(df2, df1.id == df2.id)

PySpark DataFrame transformations are a lot better in 3.3

💡 Problem statement: Create a function that classifies individuals as children, teenagers, or adults

1: Here's how you can define this transformation function in PySpark 3.3

```
def with_life_stage2(df, col_name):  
    col = F.col(col_name)  
    return df.withColumn(  
        "life_stage",  
        F.when(col < 13, "child")  
        .when(col.between(13, 19), "teenager")  
        .when(col > 19, "adult"),  
    )
```

2: Here's how to invoke the transformation

```
df.transform(with_life_stage2, "age").show()
```

```
+---+-----+-----+-----+  
| id|first_name|age|life_stage|  
+---+-----+-----+-----+  
|  8|      Liam|  4|      child|  
|  9|      Colin| 15|    teenager|  
+---+-----+-----+-----+
```

3: You needed to use nested functions in PySpark 3.2 & earlier

```
def with_life_stage(col_name):  
    def _(df):  
        col = F.col(col_name)  
        return df.withColumn(  
            "life_stage",  
            F.when(col < 13, "child")  
            .when(col.between(13, 19), "teenager")  
            .when(col > 19, "adult"),  
        )  
    return _
```

➔ 😞 Gross syntax 🛑

Apache Spark DataFrames: Transformaciones

La utilización general de los métodos de la API sigue el siguiente formato: `objetoDataFrame.nombreDelMétodo(argumentos)`. Todas las manipulaciones devuelven como resultado un nuevo DataFrame, sin modificar el original (recordamos que los RDD son inmutables **y, por extensión, los DataFrames también**). Por eso, se suelen encadenar transformaciones:

```
df.método1(args1).método2(args2)
```

Las transformaciones están contenidas en la librería `pyspark.sql.functions`. Por tanto, es necesario importar esta librería antes de usar cualquiera de estas funciones.

- **printSchema** imprime el esquema del DataFrame. Esta función es muy útil cuando queremos comprobar que el DataFrame se ha leído de fichero con el tipo de datos esperado.

```
df.printSchema()
```

- **col("nombreCol")** sirve para seleccionar una columna y devuelve un objeto `Column` sobre el que podemos realizar diferentes transformaciones. Es importante tener en cuenta que no se puede mezclar código SQL (que devuelve DataFrames) con objetos de tipo `Column`.

```
nomCliente = df.nombre nomCliente = df["ProductID"] nomCliente = col("ProductID")
```

Apache Spark DataFrames: Columns

COLUMN FUNCTION	FUNCTION DESCRIPTION
<code>alias(*alias, **kwargs)</code> <code>name(*alias, **kwargs)</code>	Provides alias to the column or expressions <code>name()</code> returns same as <code>alias()</code> .
<code>asc()</code> <code>asc_nulls_first()</code> <code>asc_nulls_last()</code>	Returns ascending order of the column. <code>asc_nulls_first()</code> Returns null values first the <code>asc_nulls_last()</code> – Returns null values after n
<code>astype(dataType)</code> <code>cast(dataType)</code>	Used to cast the data type to another type. <code>astype()</code> returns same as <code>cast()</code> .
<code>between(lowerBound, upperBound)</code>	Checks if the columns values are between lower Returns boolean value.
<code>bitwiseAND(other)</code>	Compute bitwise AND, OR & XOR of this expr

`bitwiseXOR(other)`

`contains(other)`

Check if String contains in another string.

`desc()`

Returns descending order of the column.

`desc_nulls_first()`

`desc_nulls_first()` -null values appear before non-null values.

`desc_nulls_last()`

`desc_nulls_last()` – null values appear after non-null values.

`startswith(other)`

String starts with. Returns boolean expression

`endswith(other)`

String ends with. Returns boolean expression

`eqNullSafe(other)`

Equality test that is safe for null values.

`getField(name)`

Returns a field by name in a StructField and by key in Map.

`getItem(key)`

Returns a values from Map/Key at the provided position.

`isNotNull()`

`isNotNull()` – Returns True if the current expression is NOT null.

`isNull()`

`isNull()` – Returns True if the current expression is null.

```
data=[("James","Bond","100",None),
      ("Ann","Varsa","200",'F'),
      ("Tom Cruise","XXX","400",""),
      ("Tom Brand",None,"400",'M')]
columns=["fname","lname","id","gender"]
df=spark.createDataFrame(data,columns)
```

Apache Spark DataFrames: Transformaciones

- **select** permite seleccionar columnas de diferentes formas:

```
import pyspark.sql.functions as F
#ambas formas de usar select devuelven el mismo resultado:
df.select("nombreColumna").show(5)
df.select(F.col("nombreColumna")).show(5)
# Crear columna con nombre diff y seleccionar esa columna,
# que es el resultado de restar el literal 18 a la columna eda
d.
# Mostrar 5 registros de la columna resultante seleccionada:
df.select((F.col("edad")-F.lit(18)).alias("diff")).show(5)
```

- **alias** le asigna un nombre a la columna sobre la que se aplica

```
import pyspark.sql.functions as F
df.select(F.col("nombreColumna").alias("nombreNuevo")).show(5)
```

- **withColumn** devuelve un nuevo DF con todas las columnas del original más una nueva columna añadida al final, como resultado de una operación entre columnas existentes que devuelve como resultado un objeto Column.

```
import pyspark.sql.functions as F
# Crea una nueva columna cuyo nombre es nombreNuevaColumna
# y que almacena la suma de los valores en las columnas c1 y c
2.
# Devuelve un objeto de tipo Column, del que mostramos 5 regis
tros:
df.withColumn("nombreNuevaColumna", F.col("c1")+F.col("c2")).s
how(5)
```

Apache Spark DataFrames: SELECT

- Select a Single & Multiple Columns from PySpark
- Select All Columns From List
- Select Columns By Index
- Select a Nested Column
- Other Ways to Select Columns

```
from pyspark.sql.types import StructType, StructField, StringType
schema = StructType([
    StructField('name', StructType([
        StructField('firstname', StringType(), True),
        StructField('middlename', StringType(), True),
        StructField('lastname', StringType(), True)
    ])),
    StructField('state', StringType(), True),
    StructField('gender', StringType(), True)
])
```

```
df2 = spark.createDataFrame(data = data, schema = schema)
df2.printSchema()
df2.show(truncate=False) # shows all columns
```

```
df2.select("name").show(truncate=False)
df2.select("name.firstname", "name.lastname").show(truncate=False)
df2.select("name.*").show(truncate=False)
```

```
import pyspark
from pyspark.sql import SparkSession
```

```
spark = SparkSession.builder.appName('SparkByExamples.com').getOrCreate()
```

```
data = [("James", "Smith", "USA", "CA"),
        ("Michael", "Rose", "USA", "NY"),
        ("Robert", "Williams", "USA", "CA"),
        ("Maria", "Jones", "USA", "FL")
]
columns = ["firstname", "lastname", "country", "state"]
df = spark.createDataFrame(data = data, schema = columns)
df.show(truncate=False)
df.select("firstname").show()
df.select("firstname", "lastname").show()
#Using Dataframe object name
df.select(df.firstname, df.lastname).show()
# Using col function
from pyspark.sql.functions import col
df.select(col("firstname"), col("lastname")).show()
data = [("James", None, "Smith", "OH", "M"),
        (("Anna", "Rose", ""), "NY", "F"),
        (("Julia", "", "Williams"), "OH", "F"),
        (("Maria", "Anne", "Jones"), "NY", "M"),
        (("Jen", "Mary", "Brown"), "NY", "M"),
        (("Mike", "Mary", "Williams"), "OH", "M")
]
```

Apache Spark DataFrames: Transformaciones

- **drop** elimina una columna.

```
import pyspark.sql.functions as F
# Ambas formas de utilizar drop dan el mismo resultado
df.drop("nombreColumna")
df.drop(F.col("nombreOtraColumna"))
```

- **withColumnRenamed** renombra una columna.

```
import pyspark.sql.functions as F
df.withColumnRenamed("nombreExistenteColumna", "nombreNuevoColumna")
```

- **when(condición, valorReemplazo1).otherwise(valorReemplazo2)** sirve para reemplazar valores de una columna según una condición que implica a esa o a otras columnas. Si no especificamos otherwise, los campos donde no se cumpla la condición se rellenarán a null. Esta función se utiliza generalmente dentro de withColumn:

```
import pyspark.sql.functions as F
df.withColumn("esMayor", F.when("edad>18", "mayor").otherwise("menor"))
```

- **filter** Cuando se utiliza filter() con condiciones de columna, normalmente se especifica la condición mediante expresiones de columna. Estas expresiones pueden implicar comparaciones, operaciones lógicas o incluso funciones aplicadas a columnas de DataFrame. En el siguiente ejemplo, estoy usando dfObject.colname para hacer referencia a los nombres de las columnas.

```
# Using equal condition
df.filter(df.state == "OH").show(truncate=False)
```

También puede utilizar la función col() para hacer referencia al nombre de la columna. Para usar esto primero, debe importar desde pyspark.sql.functions import col

```
# Using SQL col() function
from pyspark.sql.functions import col
df.filter(col("state") == "OH") \
    .show(truncate=False)
```

Se puede filtrar filas del DataFrame con expresiones SQL.

```
# Using SQL Expression
df.filter("gender == 'M'").show()

# For not equal
df.filter("gender != 'M'").show()
df.filter("gender <> 'M'").show()
```


Spark DataFrames: Transformaciones estadísticas

- **Columna de números aleatorios:**

```
# Crea una nueva columna con números aleatorios de una uniform  
e:  
df = df.withColumn("unif", F.rand())  
# Crea una nueva columna con números aleatorios de una normal:  
df = df.withColumn("norm", F.randn())
```

- **DataFrame con estadísticos descriptivos (media, desviación típica, máximo, mínimo, percentiles...) (método describe):**

```
from pyspark.sql import SparkSession  
from pyspark.sql.functions import col, mean, stddev, min, max, count
```

```
# Crear una SparkSession  
spark = SparkSession.builder.appName("EstadisticasDescriptivas").getOrCreate()
```

```
# Crear un DataFrame de ejemplo  
data = [("Alice", 30), ("Bob", 25), ("Charlie", 35)]  
df = spark.createDataFrame(data, ["name", "age"])
```

```
# Calcular estadísticas  
df.select(  
    mean(col("age")).alias("media_edad"),  
    stddev(col("age")).alias("desviacion_estandar_edad"),  
    min(col("age")).alias("edad_minima"),  
    max(col("age")).alias("edad_maxima"),  
    count("*").alias("total_personas")  
)show()
```

Funciones Matemáticas en Spark DataFrames

Spark DataFrames ofrecen una amplia gama de funciones para realizar operaciones matemáticas. Estas funciones se pueden aplicar a columnas individuales o a todo el DataFrame. Algunas de las más comunes son:

•Operaciones aritméticas básicas:

- **Suma, resta, multiplicación, división:** Se pueden realizar operaciones aritméticas entre columnas o entre una columna y un valor constante.
- **Módulo:** Obtener el resto de una división.
- **Potencia:** Elevar una columna a una potencia.

•Funciones trigonométricas:

- **Seno, coseno, tangente:** Calcular las funciones trigonométricas de los valores de una columna.

•Funciones exponenciales y logarítmicas:

- **Exponencial, logaritmo natural, logaritmo en base 10:** Realizar operaciones exponenciales y logarítmicas.

•Funciones de redondeo:

- **Redondear hacia arriba, redondear hacia abajo, truncar:** Ajustar los valores numéricos a un número entero.

•Funciones estadísticas:

- **Media, mediana, desviación estándar, varianza:** Calcular medidas de tendencia central y dispersión.
- **Mínimo, máximo:** Encontrar los valores mínimo y máximo de una columna.
- **Cuartiles, percentiles:** Dividir los datos en partes iguales.

•Funciones de agregación:

- **Suma, cuenta, promedio:** Realizar cálculos sobre grupos de datos.

Spark DataFrames: Transformaciones matemáticas

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sqrt, log, round

# Crear una SparkSession
spark =
SparkSession.builder.appName("OperacionesMatematicas").getOrCreate()

# Crear un DataFrame de ejemplo
data = [(1, 2.5, 3), (4, 5.2, 6.1)]
df = spark.createDataFrame(data, ["col1", "col2", "col3"])

# Realizar operaciones matemáticas
df = df.withColumn("raiz_cuadrada_col2", sqrt(col("col2")))
df = df.withColumn("logaritmo_natural_col3", log(col("col3")))
df = df.withColumn("redondeo_col2", round(col("col2")))

df.show()
```

Otro grupo de funciones son las relacionadas con la combinación y filtrado de DataFrames. Entre ellas, podemos encontrar:

- **Union de DataFrames**

```
df3 = df1.unionAll(df2)
```

- **Diferencia de DataFrames:**

```
df3 = df1.except(df2)
```

Filtrado de filas de un DataFrame. Existen dos formas de expresar la condición de filtrado:

```
import pyspark.sql.functions as F
# df3 tendrá las filas de df donde la columna c1 tenga un valor
# mayor que c2
df3 = df.where(F.col("c1")>F.col("c2"))
df3 = df.where("c1 > c2") # Es equivalente a lo anterior
```

El método `groupBy("nombreCol1", "nombreCol2", ...)` sobre un `DataFrame` devuelve una estructura de datos llamada `RelationalGroupedDataset`, que no es un `DataFrame` y sobre la que apenas se pueden aplicar operaciones. Equivale a la operación `GROUP BY` de SQL, donde se definen grupos para después calcular, para cada uno, un resultado agregado (por ejemplo, la suma, la media, un conteo, etc.) de una o varias variables numéricas.

Se suelen aplicar operaciones como `count()`, que efectúa un conteo del número de elementos de cada grupo, o la función `agg()`, que es la más habitual y realiza, para cada grupo, las agregaciones que le indiquemos sobre las columnas seleccionadas. El resultado solamente contendrá aquellas columnas que se incluyeron como argumentos en el `groupBy` más aquellas que sean mencionadas como argumento de alguna de las operaciones de agregación que incluimos en la función `agg`.

```
import pyspark.sql.functions as F  
newDF = myDF.agg(max(F.col("mycol"))) # devuelve DF de una sola fila  
newDF = myDF.groupBy("mycol").agg(F.max(F.col("mycol"))) #Tantas filas  
# como valores distintos en mycol  
newDF = myDF.withColumn("complicated", # F.sin: seno. F.lit: constante  
F.lit(2)*F.sin(F.col("colA"))*F.sqrt(F.col("colB")))  
newDF = myDF.groupBy("id").agg(F.count("id").alias("countId"),  
F.max("date").alias("maxdate"),
```

- **Sum en DataFrames** retorna la suma de todos los valores en la expresión

```
df = spark.range(10)
df.select(sum(df["id"])).show()
```

- **Avg en Dataframes** devuelve el promedio de los valores de un grupo.

```
df = spark.range(10)
df.select(avg(col("id"))).show()
```

- Las funciones de **lit()** y **typedLit()** se utilizan para agregar una nueva columna al DataFrame asignando un valor literal o constante.

```
df = spark.range(1)
df.select(lit(5).alias('height'), df.id).show()
```

- La función **split()** divide una cadena, toma la columna DataFrame de tipo String como primer argumento y el delimitador de cadena como segundo argumento en el que desea dividir.

```
df = spark.createDataFrame([('oneAtwoBthreeC',)], ['s',])
df.select(split(df.s, '[ABC]', 2).alias('s')).collect()
```

Spark DataFrames: SQL Functions

- approx count distinct
- avg
- collect list
- collect set
- countDistinct
- count
- grouping
- first
- last
- kurtosis
- max
- min
- mean
- skewness
- stddev
- stddev_samp
- stddev_pop
- sum
- sumDistinct
- variance, var_samp, var_pop

SQL Built-in Functions in Spark - Spark By {Examples}

```
# avg
print("avg: " + str(df.select(avg("salary")).collect()[0][0]))

# Prints avg: 3400.0
```

```
print("count: "+str(df.select(count("salary")).collect()[0]))

Prints county: 10
```

```
# first
df.select(first("salary")).show(truncate=False)

+-----+
|first(salary, false)|
+-----+
|3000                |
+-----+
```

3 / Apache Spark RDD

Un RDD es una colección no ordenada de objetos, distribuida en la memoria RAM de los nodos del clúster.

Es la **principal abstracción de datos**, el tipo de dato básico que tiene Apache Spark.

¿Qué es un RDD?

- RDD significa *Resilient Distributed Dataset* (Conjunto de datos distribuido y resiliente).
- Es la estructura fundamental de datos en Spark, diseñada para operar sobre clústeres distribuidos.

Características clave:

- **Inmutabilidad:** Los RDD son inmutables; cualquier operación genera un nuevo RDD.
- **Distribución:** Los datos están distribuidos a través de los nodos del clúster.
- **Tolerancia a fallos:** Spark puede recrear un RDD en caso de pérdida de datos usando el DAG (Directed Acyclic Graph) de operaciones.

Ventajas de RDD frente a MapReduce tradicional:

- Programación más sencilla gracias a operaciones funcionales como map, filter y reduce.
- Eficiencia en la ejecución mediante la evaluación perezosa (lazy evaluation).

Un RDD es una colección no ordenada de objetos, distribuida en la memoria RAM de los nodos del clúster.

Es la **principal abstracción de datos**, el tipo de dato básico que tiene Apache Spark.

- **Resilient** ('resistente', 'adaptable'): es posible reconstruir un RDD que estaba en memoria, a pesar de que una de las máquinas falle, gracias al DAG de ejecución (*directed acyclic graph*, el grafo de ejecución), que veremos más adelante.
- **Distributed** ('distribuido'): los objetos de la colección están divididos en particiones que están distribuidas en la memoria principal de los nodos del clúster. La colección no está ordenada, por lo que no se puede acceder mediante una posición a objetos individuales.
- **Dataset**: la colección representa un conjunto de datos que estamos procesando de forma paralela y distribuida, para transformarlos, calcular agregaciones, etc.

Cuando desarrollas una aplicación de Spark, estas creando un DAG. Y un DAG básicamente es un Grafo Acíclico Dirigido (**Directed Acyclic Graph**) que consta de Vértices y Aristas, donde los vértices en Spark son RDDs y las aristas representan transformaciones que se aplican a los RDD.

Un RDD es una colección no ordenada de objetos, distribuida en la memoria RAM de los nodos del clúster.

Creación de un RDD

- Desde colecciones en memoria: Ejemplo con un array en Python.
- Desde sistemas de almacenamiento externo: Ejemplo con HDFS o S3.

```
from pyspark import SparkContext
```

```
sc = SparkContext("local", "RDD Example")
```

```
# RDD desde una lista
```

```
data = [1, 2, 3, 4, 5]
```

```
rdd = sc.parallelize(data)
```

```
# RDD desde un archivo
```

```
file_rdd = sc.textFile("hdfs://path/to/file.txt")
```

¿Qué es la evaluación perezosa y como funciona?

Imagina que estás preparando una gran fiesta y tienes una lista de tareas: comprar comida, inflar globos, poner música, etc. En lugar de hacer cada tarea en el momento en que la piensas, puedes crear una lista de todas las tareas y luego ejecutarlas una por una cuando sea el momento adecuado.

La evaluación perezosa en Spark funciona de manera similar.

- **Lista de tareas:** En Spark, en lugar de una lista de tareas, tenemos una serie de operaciones que queremos realizar sobre nuestros datos.
- **Ejecución diferida:** Estas operaciones no se ejecutan inmediatamente. Spark las guarda en una especie de "lista de espera" hasta que le indicamos que realmente queremos ver los resultados.

¿Por qué es útil?

- **Optimización:** Antes de ejecutar todas las operaciones, Spark puede analizar la lista y encontrar la forma más eficiente de realizarlas todas juntas, ahorrando tiempo y recursos.
- **Flexibilidad:** Puedes cambiar la lista de operaciones sobre la marcha, sin tener que volver a empezar desde el principio.
- **Tolerancia a fallos:** Si algo sale mal durante la ejecución, Spark puede volver a ejecutar solo la parte que falló, sin tener que repetir todo el proceso.

La evaluación perezosa en Spark es como tener un asistente que toma nota de todas las cosas que quieres hacer con tus datos y luego las ejecuta de la manera más inteligente posible, solo cuando tú se lo pides. Esto hace que trabajar con grandes conjuntos de datos sea mucho más eficiente y fácil.

Evaluación perezosa (Lazy Evaluation) de los RDD:

Imagina que estás preparando una ensalada.

Evaluación tradicional (no perezosa):

- **Cortar la lechuga:** Lo haces inmediatamente, aunque luego decidas no usarla.
- **Picar el tomate:** También lo haces al instante, aunque quizás cambies de opinión y prefieras otro ingrediente.

Evaluación perezosa (como en Spark):

- **Decidir qué ingredientes usar:** Primero piensas en todos los ingredientes que podrías poner en la ensalada.
- **Cortar los ingredientes:** Solo cortas los que finalmente decides usar. Si cambias de opinión, no has perdido tiempo cortando cosas innecesarias.

¿Cómo se aplica esto en Spark?

- Spark no ejecuta un cálculo hasta que es estrictamente necesario.
- Las transformaciones (map, filter, etc.) crean un plan de ejecución, pero no se ejecutan de inmediato.
- La ejecución se dispara solo cuando se aplica una acción (count, collect, etc.).

Ventajas de la evaluación perezosa:

- **Optimización:** Spark puede optimizar el plan de ejecución antes de ejecutarlo, lo que mejora el rendimiento.
- **Flexibilidad:** Puedes cambiar el plan de ejecución sobre la marcha, sin tener que volver a calcular todo desde el principio.
- **Tolerancia a fallos:** Si una parte del cálculo falla, solo se vuelve a calcular esa parte, no todo el proceso.

Tolerancia a fallo de los RDD

Spark guarda el linaje (historial de transformaciones) de los datos para recomponer el RDD en caso de pérdida de particiones

Evaluación perezosa (Lazy Evaluation) de los RDD:

```
rdd = sc.parallelize([1, 2, 3, 4, 5])  
new_rdd = rdd.map(lambda x: x*2).filter(lambda x: x > 4)  
result = new_rdd.collect()
```

- Creas un RDD (conjunto de datos resistente y distribuido).
- Aplicas transformaciones (multiplicar por 2, filtrar los mayores que 4).
- Solo cuando haces collect() (recolectar los resultados) se ejecuta todo el cálculo.

La evaluación perezosa en Spark te permite definir una secuencia de operaciones sobre tus datos sin ejecutarlas inmediatamente. Esto te da más flexibilidad y eficiencia, especialmente cuando trabajas con grandes conjuntos de dato

¿Qué es un DAG en Spark?

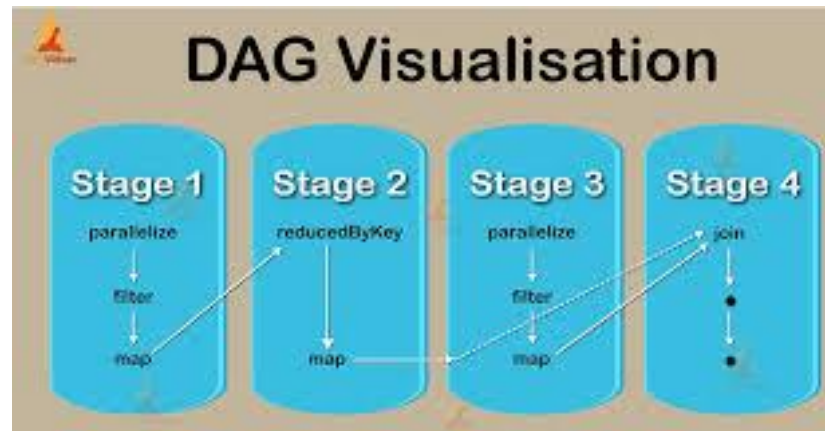
Un DAG en Spark es una representación gráfica de las transformaciones que se aplican a un RDD (Resilient Distributed Dataset). Es como un mapa de ruta que detalla cómo se procesan los datos desde su origen hasta el resultado final. Imagina un DAG como una receta de cocina: cada paso (transformación) te lleva más cerca del plato final.

Componentes de un DAG:

- **Nodos:** Representan los RDDs, es decir, los conjuntos de datos que se generan en cada etapa del procesamiento.
- **Aristas:** Representan las transformaciones que se aplican a los RDDs para obtener nuevos RDDs. Estas aristas tienen una dirección, de ahí el término "dirigido".

¿Por qué es importante el DAG?

- **Optimización:** Spark analiza el DAG antes de ejecutarlo para encontrar la forma más eficiente de procesar los datos. Esto incluye:
 - **Fusionar etapas:** Combinar varias transformaciones en una sola etapa para reducir la sobrecarga.
 - **Reducir la cantidad de datos que se mueven entre los nodos:** Optimizando el reparto de datos.
- **Ejecución en paralelo:** El DAG permite a Spark dividir el trabajo en tareas más pequeñas que pueden ejecutarse en paralelo en diferentes nodos del cluster.
- **Tolerancia a fallos:** Si una tarea falla, Spark puede volver a ejecutar solo la parte afectada del DAG, sin tener que empezar desde el principio.



¿Cómo funciona un DAG en la práctica?

1. Creación del DAG:

- Cuando escribes código Spark y aplicas transformaciones a los RDDs, estás construyendo el DAG de forma implícita.
- Cada transformación genera un nuevo nodo en el DAG y una arista que conecta el nodo anterior con el nuevo.

2. Optimización del DAG:

- El motor de ejecución de Spark analiza el DAG y aplica una serie de reglas de optimización para mejorar el rendimiento.

3. Ejecución del DAG:

- El DAG se divide en etapas. Cada etapa representa un conjunto de tareas que se pueden ejecutar en paralelo.
- Los executors ejecutan las tareas asignadas y envían los resultados al siguiente nodo del DAG.

4. Acción:

- Una acción (como count, collect, saveAsTextFile) desencadena la ejecución completa del DAG)

```
rdd = sc.parallelize([1, 2, 3, 4, 5])  
rdd_multiplicado = rdd.map(lambda x: x * 2)  
rdd_filtrado = rdd_multiplicado.filter(lambda x: x > 4)  
resultado = rdd_filtrado.collect()
```

En este ejemplo, el DAG tendría tres nodos:

- Nodo 1: RDD original
- Nodo 2: RDD multiplicado por 2
- Nodo 3: RDD filtrado

Y dos aristas:

- Arista 1: De nodo 1 a nodo 2 (transformación map)
- Arista 2: De nodo 2 a nodo 3 (transformación filter)

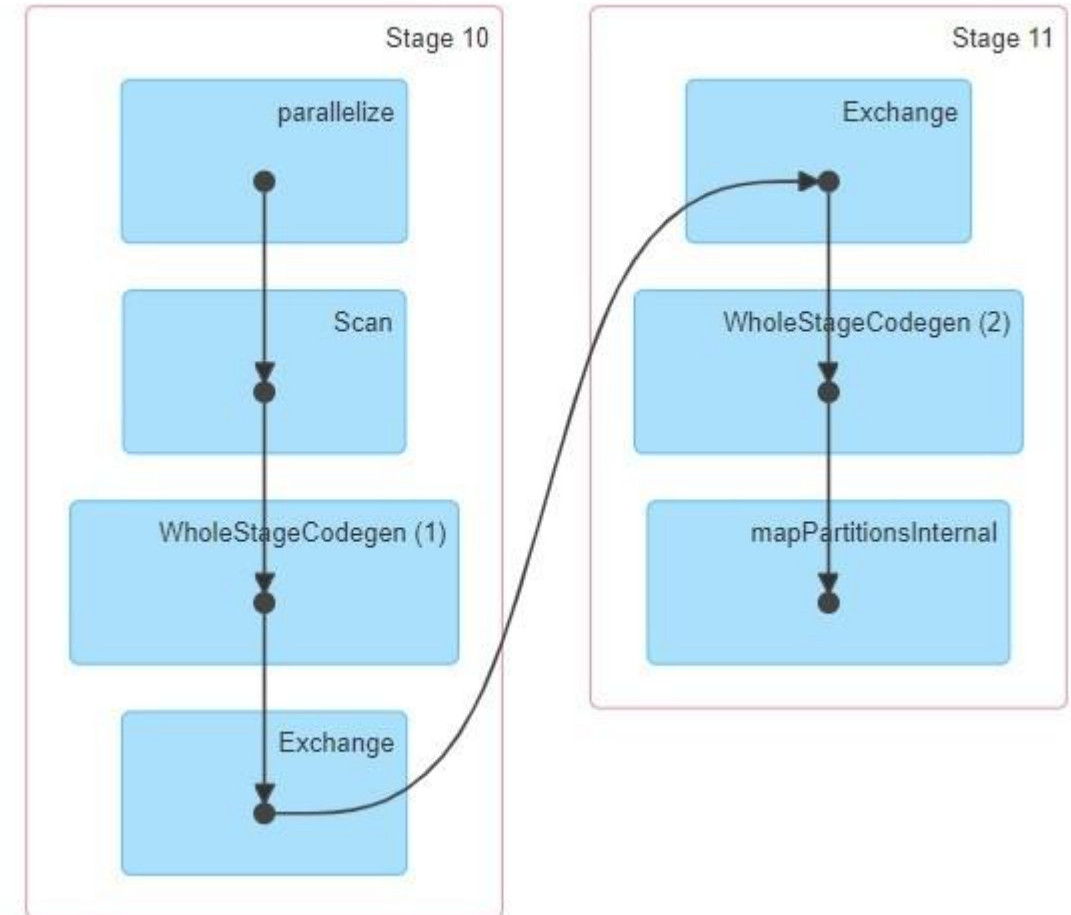
Por qué un DAG en Spark es acíclico?

Un DAG (Grafo Acíclico Dirigido) en Spark es acíclico porque **no permite ciclos o bucles en su estructura**. Esto significa que una vez que un dato ha sido procesado y transformado en un nodo del DAG, no puede volver a ese mismo nodo o a un nodo anterior.

¿Por qué es importante esta característica?

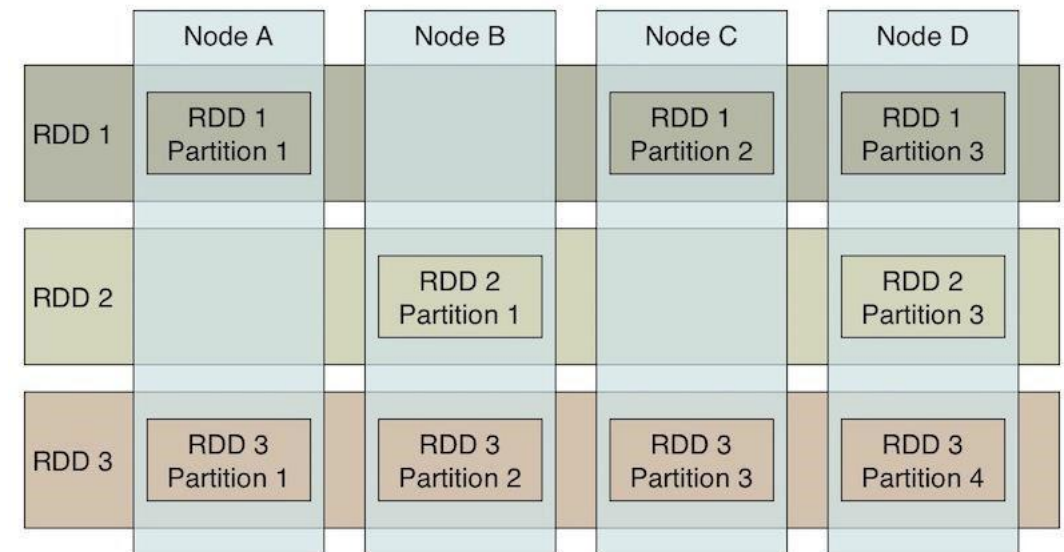
- **Orden de ejecución definido:** La ausencia de ciclos garantiza un orden de ejecución definido y predecible para las transformaciones. Cada transformación se aplica una sola vez, evitando bucles infinitos o resultados inconsistentes.
- **Optimización:** La naturaleza acíclica facilita la optimización del DAG. Spark puede analizar el grafo y determinar la secuencia más eficiente para ejecutar las transformaciones, identificando oportunidades para fusionar etapas o reducir la cantidad de datos que se mueven entre los nodos.
- **Tolerancia a fallos:** Si una tarea falla durante la ejecución, Spark puede reintentar solo la parte afectada del DAG, sin necesidad de volver a ejecutar todo el proceso desde el principio. Esto se debe a que cada nodo del DAG representa un estado intermedio del procesamiento, y se puede reanudar a partir de ese estado si es necesario.

▼ DAG Visualization



- Los RDD están particionados en los distintos nodos del clúster, ya que Apache Spark se suele instalar en un clúster o conjunto de máquinas, por lo que esos **RDDs se encuentran distribuidos sobre esas máquinas**. Con ello se consigue la tolerancia a fallos, porque si falla una máquina tenemos el fichero en otras máquinas.

La siguiente figura representa tres RDD diferentes, distribuidos en la memoria RAM de un clúster de cuatro nodos. No todos los RDD presentan el mismo número de particiones. En la figura, uno de los RDD tiene solo dos, otro tiene tres y otro, cuatro. La idea es similar al almacenamiento de los ficheros en HDFS, donde están distribuidos entre los discos duros de los nodos, con la diferencia de que, en este caso, los RDD están distribuidos en la memoria RAM de los nodos y, además, **no hay replicación de cada partición**.



- Usan la evaluación perezosa, que consiste en que **todas las transformaciones que vayamos haciendo a los RDDs se van a ir almacenando en un DAG y no se van a resolver hasta que no quede más remedio**, hasta que la herramienta esté obligada a realizarlas. Esta evaluación perezosa tiene una ventaja y un inconveniente, la primera es que ganamos tiempo, y el inconveniente es que si falla, no lo vamos a ver hasta que se resuelva el grafo completo.

En relación con los RDD, cabe destacar dos conceptos:

- **Inmutabilidad:** el contenido de un RDD no puede modificarse una vez creado. Lo que hacemos es aplicar transformaciones a los RDD para obtener otros nuevos, pero los datos del RDD existente no se alteran. La idea de la ejecución es que, cuando aplicamos una transformación, se ejecuta en paralelo sobre todas las particiones del RDD, de manera transparente al programador, para dar lugar a un nuevo RDD, cuyas particiones son el resultado de aplicar la transformación sobre cada una de las particiones del original.

***Ejemplo:** dado un RDD de números reales, para multiplicar cada elemento por dos, aplicamos una transformación que actúa en cada elemento y lo multiplica por dos. Spark lleva nuestro código de la transformación (lo serializa y lo envía por la red) a cada uno de los nodos del clúster donde haya particiones de ese RDD y lo ejecuta en ellos para que actúe en cada elemento de esa partición. Todo de manera transparente al programador.*

- **Partición:** subconjunto de los objetos de un RDD que están presentes en un mismo nodo. **Es la unidad de datos mínima sobre la que se ejecuta una tarea de transformación de manera independiente al resto de particiones.** Idealmente, tendría que haber, al menos, tantas particiones como cores físicos (procesadores) disponibles en nuestro clúster. De esta manera, nos aseguramos de que todos los cores estarán ocupados, incluso en el caso de que nada más que nuestra aplicación estuviese empleando ese clúster. Lo habitual es que haya muchas particiones de un mismo RDD en cada nodo, en un número muy superior al de procesadores existentes en ellos. Se recomienda que cada RDD esté dividido en un número de particiones que sea entre el doble y el triple que el número de procesadores del clúster.



[¿Qué es RDD \(Resilient Distributed Datasets\)? \(keepcoding.io\)](https://keepcoding.io)

■ RDD: Ciclo de Vida



Apache Spark RDD



Operaciones de transformación:

- Generan nuevos RDDs a partir de uno existente. Son evaluadas de forma perezosa.

map(func)

Aplica una función a cada elemento del RDD.

```
rdd.map(lambda x: x * 2).collect()
```

filter(func)

Filtra los elementos que cumplen con una condición.

```
rdd.filter(lambda x: x % 2 == 0).collect()
```

flatMap(func)

Similar a map, pero los resultados son "aplanados".

```
rdd.flatMap(lambda x: (x, x**2)).collect()
```

distinct()

Elimina duplicados.

```
from pyspark import SparkContext
```

```
# Crear un SparkContext
```

```
sc = SparkContext("local", "Distinct Example")
```

```
# Lista de números con duplicados
```

```
data = [1, 2, 2, 3, 4, 4, 5, 6, 6, 6]
```

```
# Crear un RDD a partir de la lista
```

```
rdd = sc.parallelize(data)
```

```
# Eliminar duplicados con distinct()
```

```
distinct_rdd = rdd.distinct()
```

```
# Recoger y mostrar el resultado
```

```
result = distinct_rdd.collect()
```

```
print("Lista sin duplicados:", result)
```

```
# Detener el SparkContext
```

```
sc.stop()
```

Operaciones de transformación:

- Generan nuevos RDDs a partir de uno existente. Son evaluadas de forma perezosa.

union(otherRDD)

Combina dos RDDs.

Supongamos que tienes dos listas de números y quieres combinarlas en un único RDD, manteniendo todos los elementos (incluidos duplicados).

```
from pyspark import SparkContext
```

```
# Crear un SparkContext
```

```
sc = SparkContext("local", "Union Example")
```

```
# Dos listas de datos
```

```
data1 = [1, 2, 3, 4]
```

```
data2 = [3, 4, 5, 6]
```

```
# Crear dos RDDs a partir de las listas
```

```
rdd1 = sc.parallelize(data1)
```

```
rdd2 = sc.parallelize(data2)
```

```
# Combinar los dos RDDs con union()
```

```
union_rdd = rdd1.union(rdd2)
```

```
# Recoger y mostrar el resultado
```

```
result = union_rdd.collect()
```

```
print("Resultado de la unión:", result)
```

```
# Detener el SparkContext
```

```
sc.stop()
```


Apache Spark RDD: Operaciones

Operaciones de acción:

Devuelven un resultado o realizan una operación específica en el RDD.

collect() Devuelve todos los elementos del RDD.

```
from pyspark import SparkContext
```

```
# Crear un SparkContext
```

```
sc = SparkContext("local", "Collect Example")
```

```
# Lista de números
```

```
data = [1, 2, 3, 4, 5]
```

```
# Crear un RDD a partir de la lista
```

```
rdd = sc.parallelize(data)
```

```
# Multiplicar cada número por 2
```

```
transformed_rdd = rdd.map(lambda x: x * 2)
```

```
# Usar collect() para traer los resultados al controlador
```

```
result = transformed_rdd.collect()
```

```
# Imprimir el resultado
```

```
print("Resultado transformado:", result)
```

```
# Detener el SparkContext
```

```
sc.stop()
```


Operaciones de acción:

Devuelven un resultado o realizan una operación específica en el RDD.

count() Cuenta el número de elementos en el RDD.

Supongamos que tienes una lista de números y quieres contar cuántos elementos contiene.

```
from pyspark import SparkContext

# Crear un SparkContext
sc = SparkContext("local", "Count Example")

# Lista de números
data = [1, 2, 3, 4, 5, 6, 7, 8, 9]

# Crear un RDD a partir de la lista
rdd = sc.parallelize(data)

# Contar el número de elementos en el RDD
element_count = rdd.count()

# Imprimir el resultado
print("Número de elementos en el RDD:", element_count)

# Detener el SparkContext
sc.stop()
```

Apache Spark RDD: Operaciones

Operaciones de acción:

Devuelven un resultado o realizan una operación específica en el RDD.

take() Devuelve los primeros n elementos del RDD.

Supongamos que tienes una lista de números y deseas recuperar los primeros 3 elementos.

```
from pyspark import SparkContext

# Crear un SparkContext
sc = SparkContext("local", "Take Example")

# Lista de números
data = [10, 20, 30, 40, 50, 60, 70]

# Crear un RDD a partir de la lista
rdd = sc.parallelize(data)

# Recuperar los primeros 3 elementos del RDD
first_three = rdd.take(3)

# Imprimir el resultado
print("Primeros 3 elementos del RDD:", first_three)

# Detener el SparkContext
sc.stop()
```

- **reduceByKey:** reduceByKey() Agrupa por clave: Toma un RDD de pares clave-valor y agrupa todos los valores asociados a una misma clave. Reduce los valores: Aplica una función de reducción (como suma, promedio, mínimo, máximo, etc.) a los valores de cada grupo para obtener un único valor por clave.

Supongamos que tenemos un RDD de pares (palabra, 1), donde cada par representa una palabra y su frecuencia (inicialmente 1). Queremos contar la frecuencia total de cada palabra:

```
from pyspark import SparkContext

sc = SparkContext.getOrCreate()

# Creamos un RDD de palabras y sus frecuencias iniciales
words = sc.parallelize([("hello", 1), ("world", 1), ("hello", 1), ("spark", 1)])

# Usamos reduceByKey para sumar las frecuencias de cada palabra
wordCounts = words.reduceByKey(lambda a, b: a + b)

# Imprimimos el resultado
print(wordCounts.collect())
```

- **sortbykey:** La función sortByKey en PySpark se utiliza para ordenar un RDD de pares clave-valor según sus claves. Es una herramienta muy útil cuando necesitas ordenar datos basados en un campo específico.

Ejemplo:

Supongamos que tenemos un RDD de pares (nombre, edad), y queremos ordenarlos por edad de mayor a menor:

```
from pyspark import SparkContext
```

```
sc = SparkContext.getOrCreate()
```

```
# Creamos un RDD de pares (nombre, edad)
```

```
people = sc.parallelize([("Alice", 30), ("Bob", 25), ("Charlie", 35)])
```

```
# Ordenamos por edad en orden descendente
```

```
peopleSorted = people.sortByKey(ascending=False)
```

```
# Imprimimos el resultado
```

```
print(peopleSorted.collect())
```

- **sample:** devuelve una muestra aleatoria del RDD del tamaño especificado como parámetro.

rdd5= sc.parallelize(range(100), 4)

6 <= rdd5.sample(False, 0.1, 81).count() <= 14

:



Pair RDDs - RDDs de clave valor en Spark